07 · Citizens directory

Everyone registered at the worker's centre, labelled by what they need, with search and filters.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The **Citizens** page lists everyone in the catchment. Each person gets one status: **attention** (open flags), **never logged**, **silent** (nothing for 14 days) or **ok**. Tabs filter by status with counts; there is a search box (name or HN reference) and a locality filter. The list is sorted by what is worth opening first.

Files involved

Only the files this feature uses.

```
backend/app/api/routes/phc.py     _directory() (the shared query), GET /phc/citizens
backend/app/services/localities.py locality_of(): empty area -> "Unspecified"
frontend/src/pages/phc/Citizens.tsx  the page
frontend/src/services/phc.ts      getDirectory()
```

How the pieces connect

1. The page keeps search, status, locality and offset in the URL.
2. `GET /phc/citizens` joins every catchment citizen with their reading and open-flag totals, works out the status, filters and sorts.
3. The response also includes counts per status (for the tabs) and the list of localities (for the dropdown).

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/api/routes/phc.py` · _directory

```
137 def _directory(worker: User, now: datetime):
138     """The catchment joined with each citizen's reading and open-flag aggregates, plus the engagement status
        ."""
139     members = catchment_ids(worker)
140     readings = (
141         select(
142             HealthRecord.user_id.label("uid"),
143             func.count().label("n"),
144             func.max(HealthRecord.timestamp).label("last"),
145         )
146         .where(HealthRecord.user_id.in_(members))
147         .group_by(HealthRecord.user_id)
148         .subquery()
149     )
150     flags = (
151         select(
152             ProcessedData.user_id.label("uid"),
153             func.count().label("n"),
154             func.max(ProcessedData.anomaly_score).label("worst"),
155         )
156         .where(ProcessedData.user_id.in_(members), *_open_flag())
157         .group_by(ProcessedData.user_id)
158         .subquery()
159     )
160     status_expr = case(
161         (func.coalesce(flags.c.n, 0) > 0, "attention"),
162         (readings.c.last.is_(None), "never_logged"),
163         (readings.c.last < now - SILENT_AFTER, "silent"),
164         else_="ok",
165     )
166     source = (
167         outerjoin(User, readings, readings.c.uid == User.id)
```

```
        .outerjoin(flags, flags.c.uid == User.id)
        .outerjoin(UserProfile, UserProfile.user_id == User.id)
    )
    return source, User.id.in_(members), status_expr, readings, flags
```

What the code means

- `readings` and `flags` are **subqueries** that summarise each citizen: number of readings and last reading time; number of open flags and worst score.
- `.subquery()` turns a query into something that can be joined like a table; its columns are reached as `readings.c.last`.
- `status_expr` is an SQL CASE: open flags give "attention", no reading ever gives "never_logged", last reading over 14 days ago gives "silent", else "ok".
- `func.coalesce(flags.c.n, 0)` treats "no row" as 0.

2. `backend/app/api/routes/phc.py` · citizens (query and sort)

```
async def citizens(
    q: str | None = Query(None, max_length=80),
    locality: str | None = Query(None, max_length=120),
    status_: Engagement | None = Query(None, alias="status"),
    limit: int = Query(50, ge=1, le=100),
    offset: int = Query(0, ge=0, le=100_000),
    worker: User = Depends(require_catchment_worker),
    db: AsyncSession = Depends(get_session),
) -> DirectoryOut:
    now = now_utc()
    source, members, status_expr, readings, flags = _directory(worker, now)
    query = select(
        User,
        status_expr.label("status"),
        func.coalesce(readings.c.n, 0),
        readings.c.last,
        func.coalesce(flags.c.n, 0),
        flags.c.worst,
        UserProfile.age,
        UserProfile.gender,
    ).select_from(source).where(members)
    if q and q.strip():
        query = query.where(_search(q))
    if locality:
        query = query.where(_locality_filter(locality))
    if status_:
        query = query.where(status_expr == status_)
    total = await _count(db, query)

    # What is worth opening first: open flags (worst first), never logged, then longest silent.
    rank = case((status_expr == "attention", 0), (status_expr == "never_logged", 1), (status_expr == "silent
        ", 2), else_=3)
    rows = await db.execute(
        query.order_by(
            rank, flags.c.worst.desc().nulls_last(), readings.c.last.asc().nulls_first(), User.created_at, U
                ser.name
        )
        .offset(offset)
        .limit(limit)
    )
    items = [
        DirectoryItem(
            citizen=person(citizen),
            age=age,
            gender=gender,
            status=state,
            open_flags=open_n,
            worst_severity=severity_of(worst) if worst is not None else None,
            readings=n,
            last_reading_at=last,
            joined_at=citizen.created_at,
        )
        for citizen, state, n, last, open_n, worst, age, gender in rows.all()
    ]
```

What the code means

- `status_: Engagement | None = Query(None, alias="status")`: the URL says `?status=`, but `status` is already used in Python, so the variable is called `status_`.
- Optional filters are added only when given.
- `rank` puts attention first, then never logged, then silent; inside a group: worst score first, then longest silent. `nulls_last()` / `nulls_first()` decide where empty values go.
- The final `for citizen, state, n, last, open_n, worst, age, gender in rows.all()` unpacks each result row into named values.

3. `frontend/src/pages/phc/Citizens.tsx` · Citizens (search and loading)

```
19  export function Citizens() {
20    const { params, set } = useQueryState();
21    const status = oneOf(params.get("status"), STATUSES, null);
22    const locality = textParam(params.get("locality"), 120);
23    const offset = intParam(params.get("offset"), 0, 0, 100_000);
24    const [search, setSearch] = useState(() => textParam(params.get("q"), 80));
25    const [q, setQ] = useState(search);
26    useEffect(() => {
27      if (search === q) return;
28      const t = window.setTimeout(() => {
29        setQ(search);
30        set({ q: search, offset: null });
31      }, 300);
32      return () => window.clearTimeout(t);
33    }, [search, q, set]);
34
35    const query = { q, locality, status, limit: PAGE, offset };
36    const dir = useResource((s) => getDirectory(query, s), JSON.stringify(query));
37    const blocked = workerState(dir);
38    if (blocked) return blocked;
```

What the code means

- `useState(() => ...)` with a function computes the starting value only once.
- The effect waits 300 ms after typing before writing `?q=` to the URL and resetting paging (`offset: null` removes it).
- `getDirectory(query, s)` is called with the URL-driven query; any change reloads.

Syntax used in this feature

Code	What it means
`query.subquery() / sub.c.col`	A query used as a table, and its columns.
`func.coalesce(x, 0)`	x, or 0 when x is empty.
`Query(None, alias="status")`	Read ?status= into a differently named variable.
`.desc().nulls_last()`	Sort descending, empty values at the end.